

Architectural Decision Record: Resolving Dataset Collisions and Repurposing 04_audit.py

1. The Core Conflict: Two Datasets, One Name

We currently face a namespace and data modeling collision:

1. **data/approved_rates_2026.csv (Hypothetical/Consumer-Level):** Modeled in 04_audit.py as per-county, per-age, per-plan-type individual consumer premiums.
2. **data/public/approved_rates_2026.csv (Real/Carrier-Level):** Generated by our bootstrap pipeline, capturing carrier-level percentage rate changes across specific rating areas (e.g., Ambetter, Highmark Inc., Partners Insurance Co.).

Is there a real per-county/age/plan rate extract from PID/Pennie?

Pennsylvania Insurance Department (PID) and Pennie (PA's health insurance exchange) publish **two distinct artifacts** during the annual rate review and open enrollment cycle:

- **Carrier Rate Filings (SERFF / Summary Approvals):** The macro-level filings showing approved average rate changes, effective dates, and primary rating area scopes (which matches our data/public/approved_rates_2026.csv).
- **Plan & Benefit Templates / QHP Rate Datasets:** Massive, highly granular tables containing every single plan ID, metal tier, and age-curve factor across all 67 counties.

If your repository's raw bootstrap data only provisions the carrier-level extension CSV (data/public/approved_rates_2026.csv), then **04_audit.py's original premise is evaluating a schema for which raw input data does not currently exist in the local workspace.**

2. Recommendations for 04_audit.py

You have three viable paths forward for 04_audit.py:

Option A: Retire 04_audit.py (Pivot to Carrier Pipeline as Source of Truth)

- **Rationale:** If Pennie/PID consumer-level rate tables are out-of-scope or unavailable in your data ingestion pipeline, keeping an audit script for a non-existent dataset creates technical debt and false test coverage.
- **Action:** Remove 04_audit.py, fold its crosswalk compatibility checks directly into 05_ingest_carrier_extensions.py, and make the carrier pipeline the definitive audit engine.

Option B: Repurpose 04_audit.py into a Unified Filing Auditor

- **Rationale:** Keep 04_audit.py, but update its schema and assertions to validate the

carrier-level extension dataset (data/public/approved_rates_2026.csv) instead of individual consumer premiums.

- **Action:** Refactor 04_audit.py to verify:
 - Carrier name consistency (e.g., ensuring Highmark Inc. and Highmark Benefits Group remain distinct).
 - Rating area range parsing correctness (1-9, semi-colon lists).
 - Decimal bounds enforcement on rate change percentages (e.g., verifying percentage adjustments fall within expected regulatory thresholds, such as -15.00\% to +35.00\%).

Option C: Rename and Partition (Keep Both Datasets Distinct)

- **Rationale:** If you expect to ingest *both* datasets in future iterations (consumer quotes vs. carrier filings):
 - Rename the consumer-level file to data/public/pennie_consumer_rates_2026.csv.
 - Keep data/public/approved_rates_2026.csv strictly for carrier-level filings.
 - Update 04_audit.py to point explicitly to the consumer file (or throw a graceful FileNotFoundError with clear instructions on how to acquire/drop the consumer extract into data/).

3. Recommended Immediate Action

Given that all 45 tests currently pass against the real carrier-level data structure, **Option B combined with renaming** is the cleanest engineering path:

1. Prevent file collision by ensuring consumer data (if ever added) uses a distinct filename.
2. Refactor 04_audit.py to run structural assertions specifically against the carrier-level extension output so that both 04_ and 05_ reinforce the same authoritative pipeline.